

Optimisation des échanges de reins

Modélisation mathématique, algorithmique sur graphes et résolution en Python

Enjeu du projet. Transformer un problème médical réel en un modèle d'optimisation capable d'identifier les échanges de reins compatibles qui maximisent le nombre - ou le bénéfice total - des greffes, tout en respectant les contraintes humaines et chirurgicales.

1. Problématique

Lorsqu'un donneur vivant souhaite aider un proche mais n'est pas compatible avec lui, un programme d'échange peut croiser plusieurs couples donneur-patient. Le défi consiste à sélectionner, dans un graphe de compatibilité, un ensemble de cycles d'échanges simultanés, disjoints et de longueur limitée. Chaque personne ne peut intervenir que dans un seul échange, et le choix final doit maximiser le poids total des greffes réalisables.

2. Objectifs

- Formaliser le Kidney Exchange Problem sous la forme d'un programme linéaire en nombres entiers (PLNE).
- Importer et structurer des instances médicales simulées sous forme de graphes orientés pondérés.
- Générer automatiquement les cycles compatibles de taille maximale k grâce à un parcours en profondeur.
- Résoudre le modèle, mesurer ses performances et vérifier la faisabilité des solutions obtenues.
- Comparer la solution principale à une borne inférieure (matching, $k = 2$) et une borne supérieure (flot, k non limité).

3. Données exploitées

ÉLÉMENT	SIGNIFICATION	UTILISATION
Sommets	Couples donneur-patient incompatibles	Unités de décision du graphe
Arcs orientés	Compatibilité d'un donneur vers un patient	Greffes potentielles
Poids	Bénéfice associé à une greffe	Fonction objectif à maximiser
Instances .wmd	16 à 2 048 couples, sans donneur altruiste	Tests de robustesse et de passage à l'échelle

Préparation des données : un parseur Python lit chaque fichier, extrait le nombre de couples et d'arcs, puis construit un dictionnaire d'adjacence. Cette structure alimente les algorithmes de génération de cycles et les modèles d'optimisation.

4. Méthode et solutions développées

01 MODÉLISATION

Variable binaire par cycle : 1 si le cycle est retenu. La fonction objectif maximise la somme des poids. Une contrainte garantit que chaque couple apparaît au plus une fois.

03 OPTIMISATION

Résolution du PLNE avec PuLP et le solveur CBC. Restitution du statut, du temps de calcul, de la valeur optimale et des cycles sélectionnés.

05 ENCADREMENT

Matching pour $k = 2$ comme borne inférieure ; modèle de flot pour k non limité comme borne supérieure. La solution du PLNE est comparée à ces deux références.

02 GÉNÉRATION DE CYCLES

Parcours en profondeur (DFS) du graphe pour détecter tous les cycles de longueur inférieure ou égale à k , avec calcul automatique de leur taille et de leur poids.

04 CONTRÔLE

Checker dédié : vérification de la taille maximale des cycles et de leur disjonction, afin de garantir une solution réellement exécutable.

06 ANALYSE

Étude du nombre de cycles, du temps de résolution et de la valeur objectif selon le nombre de couples et la taille maximale k .

5. Résultats et enseignements

- Les solutions obtenues sont cohérentes : les cycles sélectionnés sont disjoints et respectent la longueur maximale fixée.
- La valeur du PLNE est correctement encadrée par le matching et le flot, ce qui renforce la confiance dans les résultats.
- Le nombre de cycles et le temps de calcul augmentent très fortement avec la taille des instances et avec k : la génération exhaustive devient le principal goulot d'étranglement.
- À partir de $k = 3$, le gain sur le nombre maximal de greffes devient généralement faible, alors que le coût de calcul continue de croître rapidement.

POINT CLÉ

Le projet montre le compromis classique entre optimalité et passage à l'échelle : une modélisation exacte fournit une solution de référence, mais les grandes instances nécessitent des formulations plus avancées ou des heuristiques.

6. Conclusion

Ce projet de fin de licence relie un enjeu de santé publique à des compétences opérationnelles en optimisation, théorie des graphes et développement Python. Il couvre toute la chaîne de valeur : compréhension du besoin, préparation des données, conception algorithmique, modélisation mathématique, résolution, validation et analyse critique des limites. La solution développée est fiable sur les instances traitées et met clairement en évidence les leviers d'amélioration pour industrialiser l'approche : génération de colonnes, méthodes heuristiques, décomposition ou solveurs plus performants.

Compétences démontrées

Python

PuLP / CBC

Graphes & DFS

PLNE

Analyse de performance